

1. Introduction

1.1 Contexte

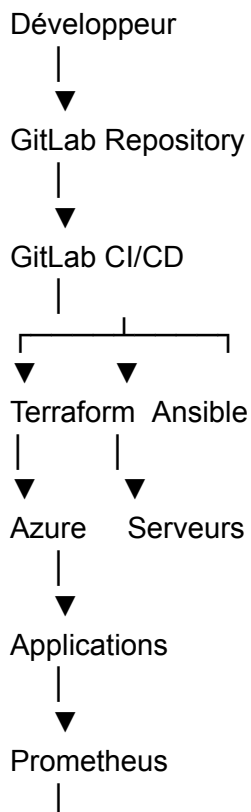
Dans le cadre du projet de modernisation du système d'information de CYNA, la mise en œuvre d'une démarche DevOps vise à automatiser le déploiement, la supervision et la maintenance des infrastructures nécessaires au fonctionnement des services de l'entreprise.

L'objectif est d'améliorer la fiabilité, la disponibilité et la rapidité de mise en production tout en réduisant les interventions manuelles et les risques d'erreurs humaines.

Cette démarche s'inscrit dans la stratégie globale de transformation numérique de CYNA et doit accompagner la croissance future de l'entreprise ainsi que le développement de ses services SaaS et de cybersécurité.

2. Architecture DevOps cible

L'architecture retenue repose sur une chaîne d'automatisation complète permettant de gérer le cycle de vie des infrastructures et des applications.





Cette architecture permet d'assurer une automatisation cohérente depuis le développement jusqu'à l'exploitation.

3. Infrastructure as Code

3.1 Objectifs

L'Infrastructure as Code (IaC) permet de décrire l'infrastructure sous forme de fichiers versionnés.

Cette approche apporte :

- reproductibilité ;
 - traçabilité ;
 - rapidité de déploiement ;
 - réduction des erreurs humaines.
-

3.2 Choix de Terraform

Terraform a été retenu pour le provisionnement des ressources Azure.

Ressources concernées :

- Resource Groups
- Réseaux virtuels
- Sous-réseaux
- Machines virtuelles
- Groupes de sécurité
- Adresses IP publiques

Avantages

- déploiement reproductible ;
- gestion d'état ;
- compatibilité Azure ;
- intégration GitLab CI/CD.

4. Configuration automatisée

4.1 Choix d'Ansible

Ansible est utilisé afin d'automatiser la configuration des systèmes après leur création par Terraform.

Le rôle d'Ansible est complémentaire à celui de Terraform :

Terraform	Ansible
Crée les ressources	Configure les ressources
Provisionne Azure	Installe les services
Réseau	Configuration système
VM	Applications

4.2 Playbooks développés

Les playbooks suivants sont prévus :

hardening_base.yml

Configuration de sécurité des serveurs :

- mises à jour ;
- Fail2ban ;
- pare-feu ;
- SSH sécurisé.

```
1      ---
2      - name: Configuration de sécurité de base
3        hosts: all
4        become: yes
5
6        tasks:
7          - name: Mettre à jour les paquets
8            apt:
9              update_cache: yes
10             upgrade: yes
11
12          - name: Installer les outils essentiels
13            apt:
14              name:
15                - curl
16                - wget
17                - git
18                - ufw
19                - fail2ban
20                - vim
21              state: present
22
23          - name: Activer le pare-feu UFW
24            ufw:
25              state: enabled
26              policy: deny
27
28          - name: Autoriser SSH
29            ufw:
30              rule: allow
31              port: "22"
32              proto: tcp
33
34          - name: Activer Fail2ban
35            service:
36              name: fail2ban
37              state: started
38              enabled: yes
```

install_monitoring.yml

Déploiement :

- Prometheus ;
- Grafana.

```
1  ---
2  - name: Déploiement Prometheus et Grafana
3    hosts: monitoring
4    become: yes
5
6    tasks:
7      - name: Créer dossier monitoring
8        file:
9          path: /opt/monitoring
10         state: directory
11
12      - name: Créer docker-compose Prometheus/Grafana
13        copy:
14          dest: /opt/monitoring/docker-compose.yml
15          content: |
16            version: '3'
17            services:
18              prometheus:
19                image: prom/prometheus
20                container_name: prometheus
21                ports:
22                  - "9090:9090"
23                volumes:
24                  - ./prometheus.yml:/etc/prometheus/prometheus.yml
25
26              grafana:
27                image: grafana/grafana
28                container_name: grafana
29                ports:
30                  - "3000:3000"
31                depends_on:
32                  - prometheus
33
34      - name: Créer configuration Prometheus
35        copy:
36          dest: /opt/monitoring/prometheus.yml
37          content: |
38            global:
39              scrape_interval: 15s
40
41            scrape_configs:
42              - job_name: 'prometheus'
43                static_configs:
44                  - targets: ['localhost:9090']
```

install_graylog.yml

Déploiement :

- MongoDB ;
- OpenSearch ;
- Graylog.

```
1  ---
2  - name: Déploiement Graylog
3    hosts: logs
4    become: yes
5
6    tasks:
7      - name: Créer dossier Graylog
8        file:
9          path: /opt/graylog
10         state: directory
11
12      - name: Créer docker-compose Graylog
13        copy:
14          dest: /opt/graylog/docker-compose.yml
15          content: |
16            version: '3'
17            services:
18              mongo:
19                image: mongo:5.0
20
21              opensearch:
22                image: opensearchproject/opensearch:2
23                environment:
24                  - discovery.type=single-node
25                  - plugins.security.disabled=true
26                  - OPENSEARCH_JAVA_OPTS=-Xms1g -Xmx1g
27
28              graylog:
29                image: graylog/graylog:5.2
30                environment:
31                  - GRAYLOG_PASSWORD_SECRET=changeme_secret
32                  - GRAYLOG_ROOT_PASSWORD_SHA2=8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918
33                  - GRAYLOG_HTTP_EXTERNAL_URI=http://localhost:9000/
34                ports:
35                  - "9000:9000"
36                  - "1514:1514"
37                  - "12201:12201/udp"
38                depends_on:
39                  - mongo
40                  - opensearch
41
42      - name: Lancer Graylog
43        command: docker-compose up -d
```

install_gitlab_runner.yml

Installation des GitLab Runners.

```
1  ---
2  - name: Installation GitLab Runner
3    hosts: devops
4    become: yes
5
6    tasks:
7      - name: Télécharger script GitLab Runner
8        get_url:
9          url: https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh
10         dest: /tmp/gitlab-runner.sh
11         mode: '0755'
12
13      - name: Ajouter dépôt GitLab Runner
14        command: /tmp/gitlab-runner.sh
15
16      - name: Installer GitLab Runner
17        apt:
18          name: gitlab-runner
19          state: present
20          update_cache: yes
21
22      - name: Activer GitLab Runner
23        service:
24          name: gitlab-runner
25          state: started
26          enabled: yes
```

backup_config.yml

Sauvegarde automatique des configurations critiques.

```
1    ---
2    - name: Sauvegarde des configurations critiques
3      hosts: all
4      become: yes
5
6      tasks:
7        - name: Créer dossier backup
8          file:
9            path: /backup/configs
10           state: directory
11
12        - name: Sauvegarder /etc
13          archive:
14            path: /etc
15            dest: "/backup/configs/etc-{{ inventory_hostname }}.tar.gz"
16            format: gz
```

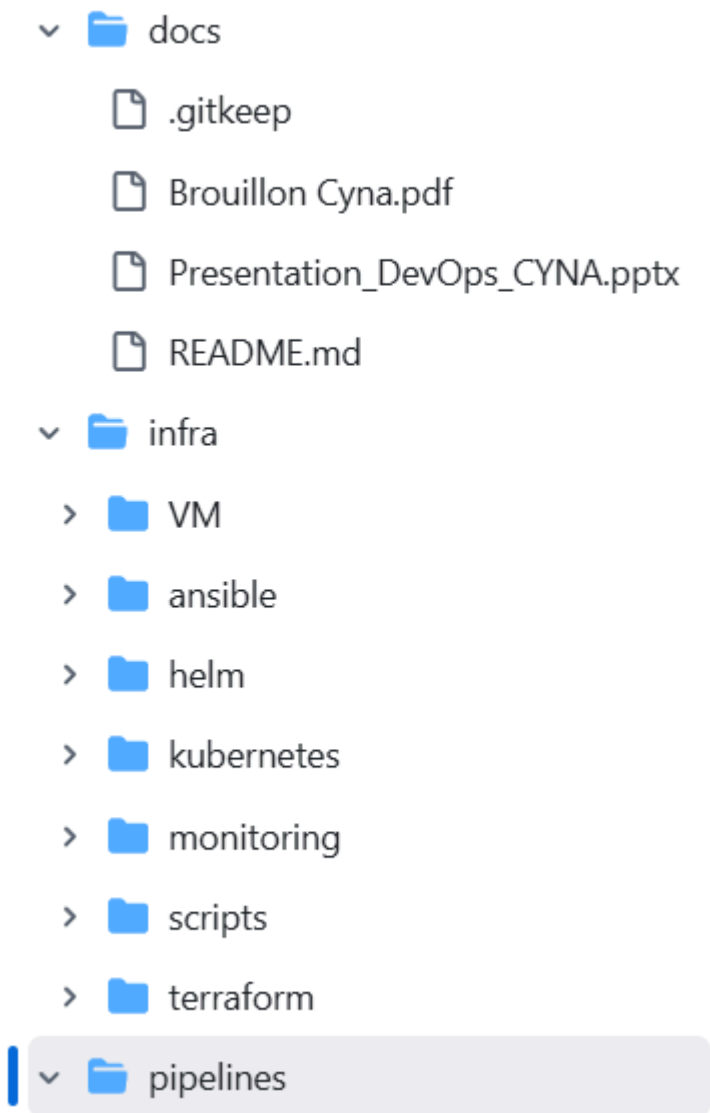
5. Gestion des versions et CI/CD

5.1 Git

L'ensemble du code d'infrastructure est stocké dans un dépôt Git unique.

Structure :

```
infra/
terraform/
ansible/
monitoring/
scripts/
docs/
pipelines/
```

5.2 GitLab CI/CD

GitLab CI/CD automatise :

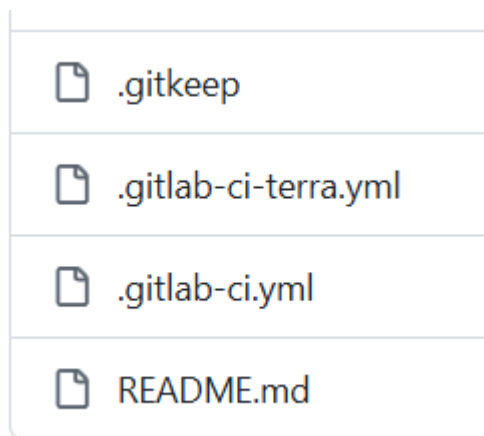
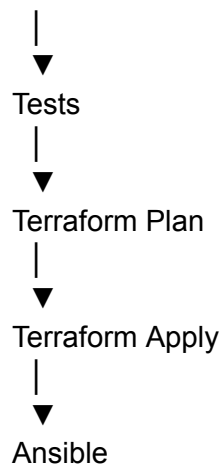
- validation Terraform ;
- tests ;
- déploiement ;
- contrôle qualité.

Pipeline :

Commit



Validation



6. Monitoring et supervision

6.1 Prometheus

Prometheus collecte les métriques :

- CPU ;
- mémoire ;
- disque ;
- disponibilité ;
- services.

6.2 Grafana

Grafana fournit :

- dashboards ;
- graphiques ;
- alertes.

Alertes prévues

Élément	Seuil
CPU	> 80 %
RAM	> 85 %
Disque	> 90 %
Service indisponible	immédiat

7. Gestion centralisée des logs

7.1 Besoin

L'infrastructure génère quotidiennement de nombreux événements :

- authentications ;
- erreurs applicatives ;
- logs systèmes ;
- événements de sécurité.

Une centralisation est indispensable.

7.2 Choix de Graylog

Graylog a été retenu afin de :

- centraliser les logs ;
- faciliter les investigations ;
- améliorer la supervision.

Sources :

- Linux ;
- Windows ;
- Firewall ;
- Azure ;
- Applications SaaS.

8. PRA / PCA

L'automatisation DevOps participe directement à la stratégie de continuité d'activité.

Les mécanismes prévus sont :

- sauvegarde automatique ;
- reconstruction de l'infrastructure via Terraform ;
- réinstallation automatique des services via Ansible ;
- restauration rapide des configurations.

Cette approche réduit significativement le temps nécessaire à la remise en service d'une plateforme après incident.

9. Livrables DevOps

Les livrables produits seront :

- dépôt Git structuré ;
 - scripts Terraform ;
 - playbooks Ansible ;
 - pipelines GitLab CI/CD ;
 - documentation technique ;
 - dashboards Grafana ;
 - plateforme Graylog ;
 - procédures PRA/PCA ;
 - schémas d'architecture.
-

10. Conclusion

La mise en œuvre de cette architecture DevOps permettra à CYNA de disposer d'un environnement moderne, automatisé et facilement maintenable. L'association de Terraform, Ansible, GitLab CI/CD, Prometheus, Grafana et Graylog garantit une meilleure maîtrise des infrastructures, une réduction des risques opérationnels et une amélioration significative de la disponibilité des services.